

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

О Т Ч Е Т

по ЛАБОРАТОРНОЙ РАБОТЕ №3.2

Тема: Реализация базы данных в PostgreSQL

Выполнили: **Абакар Иссака Малли К3241**

Преподаватель : **Говорова Марина Михайловна**

Дата **12.04.2026**

Санкт-Петербург 2026

Введение

В данной лабораторной работе была выполнена практическая реализация базы данных на основе ранее разработанных инфологической и даталогической моделей. Работа направлена на закрепление навыков проектирования и создания баз данных с использованием системы управления базами данных PostgreSQL и инструмента pgAdmin 4.

Цель работы:

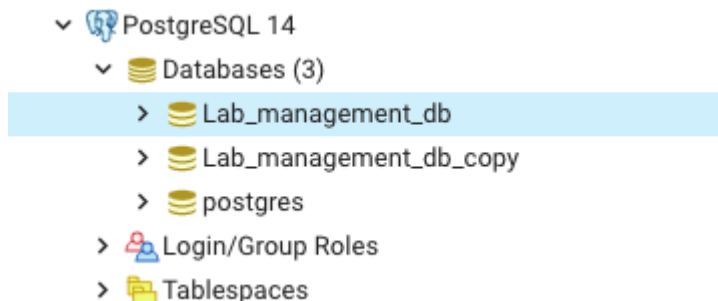
Основной целью является реализация структуры базы данных, соответствующей предметной области, а также получение практических навыков работы с SQL, создания таблиц, связей, выполнения резервного копирования и восстановления данных.

Задачи:

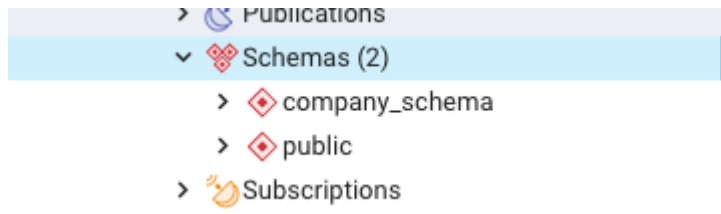
1. Создать базу данных в PostgreSQL.
2. Создать схему для организации объектов.
3. Реализовать таблицы на основе лабораторной работы №2.
4. Настроить связи между таблицами с помощью внешних ключей.
5. Добавить тестовые данные.
6. Выполнить резервное копирование базы данных.
7. Выполнить восстановление базы данных.

Ход работы:

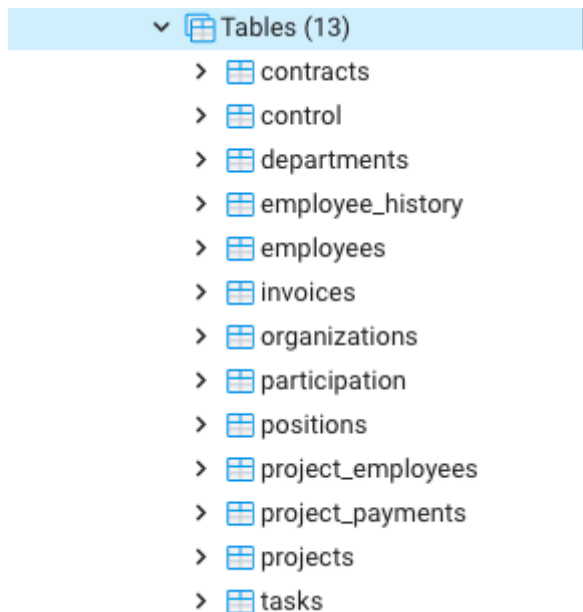
1. Создание базы данных. В среде pgAdmin 4 была создана база данных с именем "Lab_management_db". Данная база данных предназначена для хранения информации о проектах, заданиях и сотрудниках.



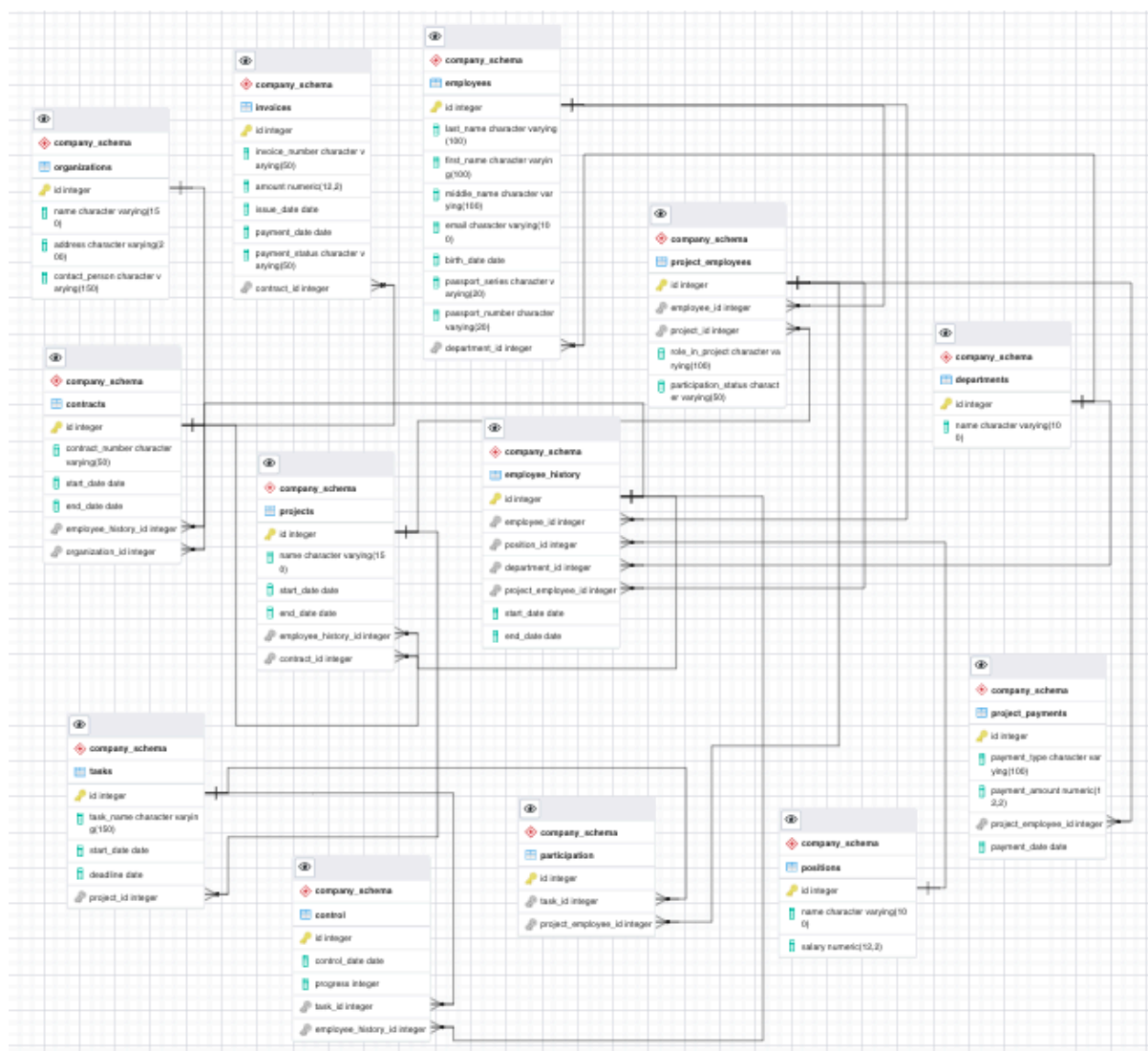
2. Создание схемы. Для логической организации объектов была создана схема "company_schema". Использование схем позволяет структурировать базу данных и упрощает управление объектами.



3. Создание таблиц. На основе лабораторной работы №2 были реализованы следующие таблицы: departments, employees, positions, employee_history, organizations, contracts, projects, tasks, participation и control. Каждая таблица содержит первичный ключ, а также необходимые атрибуты согласно модели данных.



4. Реализация связей. Связи между таблицами реализованы с помощью внешних ключей (Foreign Key). Это обеспечивает целостность данных и позволяет корректно связывать сущности между собой. Например, таблица tasks связана с projects, а participation связывает сотрудников и задания.



5. Добавление данных. В таблицы были добавлены тестовые данные с использованием SQL-запросов INSERT. Это позволило проверить корректность структуры базы данных и связей между таблицами.

6. Резервное копирование. Была выполнена операция резервного копирования базы данных в двух форматах: custom (.backup) и plain SQL (.sql). Это позволяет сохранить структуру и данные базы.

7. Восстановление базы данных. Была создана новая база данных, в которую выполнено восстановление из файла резервной копии. После восстановления была проверена корректность структуры и данных.

Реализация базы данных в PostgreSQL

Создание структуры базы данных:

Структура базы данных была реализована с помощью SQL-запросов CREATE TABLE. Ниже приведён основной код создания таблиц.

```
CREATE TABLE company_schema.project_employees (  
    id SERIAL PRIMARY KEY,  
    employee_id INTEGER,  
    project_id INTEGER,  
    role_in_project VARCHAR(100),  
    participation_status VARCHAR(50)  
);
```

```
CREATE TABLE company_schema.employee_history (  
    id SERIAL PRIMARY KEY,  
    employee_id INTEGER,  
    position_id INTEGER,  
    department_id INTEGER,  
    project_employee_id INTEGER,  
    start_date DATE,  
    end_date DATE  
);
```

```
CREATE TABLE company_schema.contracts (  
    id SERIAL PRIMARY KEY,  
    contract_number VARCHAR(50) NOT NULL,  
    start_date DATE,  
    end_date DATE,  
    employee_history_id INTEGER,  
    organization_id INTEGER  
);
```

```
CREATE TABLE company_schema.projects (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    start_date DATE,  
    end_date DATE,  
    employee_history_id INTEGER,  
    contract_id INTEGER  
);
```

```
CREATE TABLE company_schema.tasks (  
    id SERIAL PRIMARY KEY,  
    task_name VARCHAR(150),  
    start_date DATE,  
    deadline DATE,  
    project_id INTEGER  
);
```

```

CREATE TABLE company_schema.participation (
    id SERIAL PRIMARY KEY,
    task_id INTEGER,
    project_employee_id INTEGER
);

CREATE TABLE company_schema.control (
    id SERIAL PRIMARY KEY,
    control_date DATE,
    progress INTEGER CHECK (progress >= 0 AND progress <= 100),
    task_id INTEGER,
    employee_history_id INTEGER
);

CREATE TABLE company_schema.invoices (
    id SERIAL PRIMARY KEY,
    invoice_number VARCHAR(50),
    amount NUMERIC(12,2),
    issue_date DATE,
    payment_date DATE,
    payment_status VARCHAR(50),
    contract_id INTEGER
);

CREATE TABLE company_schema.project_payments (
    id SERIAL PRIMARY KEY,
    payment_type VARCHAR(100),
    payment_amount NUMERIC(12,2),
    project_employee_id INTEGER,
    payment_date DATE
);

```

Foreign Keys:

```

-- employees -> departments
ALTER TABLE company_schema.employees
ADD CONSTRAINT fk_employees_department
FOREIGN KEY (department_id)
REFERENCES company_schema.departments(id);

-- project_employees -> employees
ALTER TABLE company_schema.project_employees
ADD CONSTRAINT fk_project_employees_employee

```

```
FOREIGN KEY (employee_id)
REFERENCES company_schema.employees(id);
```

```
-- project_employees -> projects
ALTER TABLE company_schema.project_employees
ADD CONSTRAINT fk_project_employees_project
FOREIGN KEY (project_id)
REFERENCES company_schema.projects(id);
```

```
-- employee_history
ALTER TABLE company_schema.employee_history
ADD CONSTRAINT fk_employee_history_employee
FOREIGN KEY (employee_id)
REFERENCES company_schema.employees(id);
```

```
ALTER TABLE company_schema.employee_history
ADD CONSTRAINT fk_employee_history_position
FOREIGN KEY (position_id)
REFERENCES company_schema.positions(id);
```

```
ALTER TABLE company_schema.employee_history
ADD CONSTRAINT fk_employee_history_department
FOREIGN KEY (department_id)
REFERENCES company_schema.departments(id);
```

```
ALTER TABLE company_schema.employee_history
ADD CONSTRAINT fk_employee_history_project_employee
FOREIGN KEY (project_employee_id)
REFERENCES company_schema.project_employees(id);
```

```
-- contracts
ALTER TABLE company_schema.contracts
ADD CONSTRAINT fk_contracts_employee_history
FOREIGN KEY (employee_history_id)
```

```
REFERENCES company_schema.employee_history(id);
```

```
ALTER TABLE company_schema.contracts  
ADD CONSTRAINT fk_contracts_organization  
FOREIGN KEY (organization_id)  
REFERENCES company_schema.organizations(id);
```

```
-- projects
```

```
ALTER TABLE company_schema.projects  
ADD CONSTRAINT fk_projects_employee_history  
FOREIGN KEY (employee_history_id)  
REFERENCES company_schema.employee_history(id);
```

```
ALTER TABLE company_schema.projects  
ADD CONSTRAINT fk_projects_contract  
FOREIGN KEY (contract_id)  
REFERENCES company_schema.contracts(id);
```

```
-- tasks
```

```
ALTER TABLE company_schema.tasks  
ADD CONSTRAINT fk_tasks_project  
FOREIGN KEY (project_id)  
REFERENCES company_schema.projects(id);
```

```
-- participation
```

```
ALTER TABLE company_schema.participation  
ADD CONSTRAINT fk_participation_task  
FOREIGN KEY (task_id)  
REFERENCES company_schema.tasks(id);
```

```
ALTER TABLE company_schema.participation  
ADD CONSTRAINT fk_participation_project_employee  
FOREIGN KEY (project_employee_id)  
REFERENCES company_schema.project_employees(id);
```

-- control

ALTER TABLE company_schema.control

ADD CONSTRAINT fk_control_task

FOREIGN KEY (task_id)

REFERENCES company_schema.tasks(id);

ALTER TABLE company_schema.control

ADD CONSTRAINT fk_control_employee_history

FOREIGN KEY (employee_history_id)

REFERENCES company_schema.employee_history(id);

-- invoices

ALTER TABLE company_schema.invoices

ADD CONSTRAINT fk_invoices_contract

FOREIGN KEY (contract_id)

REFERENCES company_schema.contracts(id);

-- project_payments

ALTER TABLE company_schema.project_payments

ADD CONSTRAINT fk_project_payments_project_employee

FOREIGN KEY (project_employee_id)

REFERENCES company_schema.project_employees(id);

Вывод

В ходе выполнения лабораторной работы была успешно реализована база данных, соответствующая ранее разработанной модели. Были созданы таблицы, настроены связи между ними, добавлены тестовые данные, а также выполнены операции резервного копирования и восстановления. Полученные навыки являются важной основой для дальнейшего изучения баз данных и их применения в реальных информационных системах.